

Insight.AI: Project Documentation

1. Overview

Insight.AI is a powerful Chrome extension that solves the problem of encountering complex or confusing text online. Instead of switching tabs to search for definitions, this tool helps you get instant understanding directly on the page. Using the Google Gemini API, you can select any text to receive a concise, AI-generated explanation, summary, or translation, integrating a smart assistant seamlessly into your browsing workflow.

Tagline: Your Smart Research Companion

2. Live Demo

A live demonstration of the project's features and functionality is available on YouTube.

[!Insight.AI Demo on YouTube](#)

3. Project Journey

This section details the story behind Insight.AI, from the initial idea to the challenges faced during development.

The Inspiration

The idea for Insight.AI was born from a simple, everyday frustration: context switching. While reading articles, technical documentation, or research papers, I constantly found myself highlighting unfamiliar terms, opening a new tab, and searching for a definition. This cycle, though small, breaks concentration and disrupts the flow of learning. I envisioned a tool that could act as an intelligent companion, living right inside the browser, to make the web a more seamless and enriching environment for curiosity and research.

How It Was Built

I chose to build this as a Chrome Extension using Manifest V3, the modern standard that emphasizes security and performance. The architecture is intentionally lightweight, relying on vanilla JavaScript, HTML, and CSS to keep the extension fast and responsive.

The project is broken down into a few key components:

1. [background.js](#) (The Brain): This service worker is the central hub. It listens for user actions (like right-clicking text), creates the context menus, and handles all communication with the Google Gemini API.
2. Content Scripts (The Hands): These scripts are the bridge to the webpage content. [tooltip_content_script.js](#) injects the floating tooltip, [history_content_script.js](#) manages the history sidebar, and the `showAIResponse` function renders the draggable AI response pop-up.
3. [popup.html](#) (The Control Panel): This is the UI for the extension's settings, allowing users to configure the theme and AI response behavior.

The "magic" comes from the Google Gemini API. I chose the `gemini-2.0-flash-001` model because it strikes an excellent balance between speed and intelligence. By crafting specific prompts for explaining, summarizing, and translating, I could leverage a single powerful model for all of the extension's core features.

Challenges & Learnings

This project was a fantastic learning experience in asynchronous JavaScript, Chrome's MV3 architecture, and defensive DOM manipulation. The biggest challenges were:

- **API Key Management:** The current method requires a user to add their key to the source code. A future improvement would be a secure options page to store the key in `chrome.storage`.
 - **State Synchronization:** Keeping the theme (dark/light) consistent across the popup, response box, and history sidebar required using `chrome.storage.sync` as a single source of truth.
 - **Race Conditions:** Early versions could trigger multiple UI popups. This was solved by ensuring any existing popup is removed before a new one is created, making the user experience clean and predictable.
-

4. Core Features

- Context-Aware Actions: Right-click on selected text to Explain, Summarize, or Translate to English.
 - Floating Tooltip: A subtle tooltip appears on text selection for quick "Explain" actions. AI
 - History Sidebar: Access, review, and reuse past queries and responses. Interactive
 - Response UI: AI responses are displayed in a sleek, draggable pop-up with a "typing" effect.
 - Customizable Settings :
 - Theme Control: Switch between a modern dark mode and a clean light mode.
 - Response Mode: Adjust the AI's verbosity (concise, detailed, simple).
 - Smart Caching: Repeated queries are served from a local cache to provide instant responses and reduce API usage.
-

5. Technology Stack

- Languages: JavaScript (ES6+), HTML5, CSS3
 - Platform: Chrome Extension APIs (Manifest V3)
 - AI Provider: Google Gemini API (generativelanguage.googleapis.com)
-

6. Code Architecture

- [background.js](#): The service worker. Handles API calls, context menu logic, caching, history management, and message passing. It is the "brain" of the extension.
 - Content Scripts ([tooltip_content_script.js](#), [history_content_script.js](#)): These scripts are injected into web pages to interact with the DOM. They are responsible for creating the floating tooltip and the history sidebar. The `showAIResponse` function is also injected from the background script to display the main response popup.
 - [popup.html](#) / [popup.js](#) / [style.css](#): These files create the UI and logic for the extension's settings popup, which appears when the user clicks the toolbar icon.
 - [manifest.json](#): The core configuration file that defines the extension's permissions, scripts, icons, and other essential metadata for the Chrome browser.
 - *.css files: These stylesheets provide the visual appearance for all UI components, including the main popup, the on-page response box, the history sidebar, and the tooltip, with full support for both dark and light themes.
-

7. Getting Started

Follow these steps to install and run the extension locally.

Prerequisites

- Google Chrome
- A Google Gemini API Key. You can get one from Google AI Studio.

Installation

1.

Clone the repository:

```
bash git clone https://github.com/kumaraguru911/insight-ai-extension.git  
cd insight-ai- extension
```

2.

Add Your API Key:

- Open the [background.js](#) file.
- Find the line that contains YOUR_API_KEY.
- Replace YOUR_API_KEY with your actual Gemini API key.

3.

Load the Extension in Chrome:

- Open Chrome and navigate to `chrome://extensions`.
- Enable "Developer mode" using the toggle in the top-right corner.
- Click the "Load unpacked" button.
- Select the insight-ai-extension directory that you cloned.

8. License

This project is licensed under the MIT License. See the [LICENSE.md](#) file for full details.